1. **Задание**

   Целью работы является реализация простой системы распределенной репликации (“писатели-читатели”).

   1 Скачайте и разверните Apache Kafka

   2 Модифицируйте свое приложение со встраиваемой базой данных так, чтобы его можно было запустить в нескольких экземплярах на разных портах

   3 Реализуйте в рамках своего приложения Producer и Consumer такие, что a. Producer при каждой операции записи оповещает соответствующий топик b. Consumer при получении информации из топика записывает обновление в локальную (встроенную в приложение) базу

   4 Продемонстрируйте, что информация, записанная одним приложением, доступна второму приложению.

2. **Описание продукта**

   Необходимо реализовать систему распределённой репликации для AJAX приложения

3. **Листинг программного кода**

   RecordProducer.java

```
@Slf4j
@Service
@RequiredArgsConstructor
public class RecordProducer {
    private final KafkaTemplate<String, String> kafkaTemplate;
    @Value("${kafka.topic}")
    private String topic;

    public void sendNewItem(GOST entity) {

        log.info("Отправили сообщение добавления");
        kafkaTemplate.send(topic, "Add" + ";" + entity.getId() + ";" + entity.getYear() + ";"
+ entity.getAuthor() + ";" + entity.getName());
    }

    public void sendDeleteItem(Long id) {
        log.info("Отправили сообщение удаления");
        kafkaTemplate.send(topic, "Delete" + ";" + id);
```

```
    }
}
```

RecordConsumer.java

```
@Slf4j
@Service
@RequiredArgsConstructor
public class RecordConsumer {
    private final GostRepository GostRep;

    @KafkaListener(topics = "${kafka.topic}", groupId =
"${spring.kafka.consumer.group-id}")
    public void consume(String message) {
        String[] parts = message.split(";");
        log.info("Приняли сообщение" + parts[0]);
        if (Objects.equals(parts[0], "Add"))
        {
            if (parts.length < 5) return;
            if (!GostRep.existsById(Long.parseLong(parts[1])))
            {
                GostRep.save(new GOSTPE(Long.parseLong(parts[1]),
Long.parseLong(parts[2]), parts[3], parts[4]));
            }
        }
        else if (Objects.equals(parts[0], "Delete"))
        {
            if (parts.length < 2) return;
            if (GostRep.existsById(Long.parseLong(parts[1])))
            {
                GostRep.deleteById(Long.parseLong(parts[1]));
            }
        }
    }
}
```

Application.properties

```
spring.jpa.hibernate.ddl-auto=update
spring.jpa.hibernate.naming.physical-strategy=org.hibernate.boot.model.naming.PhysicalNamingStrategyStandardImpl
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
spring.jpa.properties.hibernate.globally_quoted_identifiers=true
spring.datasource.url=jdbc:h2:mem:testdb;MODE=PostgreSQL
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=password
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.jpa.defer-datasource-initialization=true

spring.application.name=lab7
server.port: 8443
server.ssl.key-store: target/keystore.p12
server.ssl.key-store-password: mypassword
server.ssl.keyStoreType: PKCS12
server.ssl.keyAlias: tomcat

kafka.bootstrap-servers=localhost:9092
spring.kafka.consumer.group-id=group-app1
kafka.topic=my-topic
```

Gost_ApiController.java

```
@Slf4j
@Generated(value = "org.openapitools.codegen.languages.SpringCodegen", date = "2025-09-21T17:15:00.750982+03:00[Europe/Moscow]", comments = "Generator version: 7.7.0")
@Controller
@RequestMapping("${openapi..base-path:}")
@RequiredArgsConstructor
public class GostApiController implements GostApi {
```

```
    private final NativeWebRequest request;

    private final GostWorker GostWorker;
    private final RecordProducer producer;

    @Override
    public Optional<NativeWebRequest> getRequest() {
        return Optional.ofNullable(request);
    }

    @Override
    public ResponseEntity<GOST> addBook(@Parameter(name = "GOST", description =
"Добавление новой литературы", required = true) @Valid @RequestBody GOST
GOSTBody, Principal principal)
    {
        if (principal == null) {
            throw new ResponseStatusException(HttpStatus.FORBIDDEN, "Доступ
запрещён");
        }
        if (GOSTBody.getId() == null) {
            return ResponseEntity.status(HttpStatus.BAD_REQUEST).build();
        }

        if (GostWorker.IsGOSTExist(GOSTBody.getId())) {
            return ResponseEntity.status(HttpStatus.CONFLICT).build();
        }

        log.info("User {} добавил литературу: {}", principal.getName(), GOSTBody);

        GostWorker.AddGOST(GOSTBody.getId(), GOSTBody.getYear(),
GOSTBody.getAuthor(), GOSTBody.getName());
        producer.sendNewItem(GOSTBody);
        return ResponseEntity.ok(GOSTBody);
    }
```

```
    @Override
    public ResponseEntity<Void> deleteBook(@NotNull @Parameter(name = "id",
description = "идентификатор литературы", required = true, in = ParameterIn.QUERY)
@Valid @RequestParam(value = "id", required = true) Long id, Principal principal)
    {
        if (principal == null) {
            throw new ResponseStatusException(HttpStatus.FORBIDDEN, "Доступ
запрещён");
        }
        if (id == null) {
            return ResponseEntity.status(HttpStatus.BAD_REQUEST).build();
        }
        if (!GostWorker.IsGOSTExist(id)) {
            return ResponseEntity.status(HttpStatus.NOT_FOUND).build();
        }

        log.info("User {} удалил литературу по ID {}", principal.getName(), id);
        GostWorker.DeleteById(id);
        producer.sendDeleteItem(id);
        return null;
    }

    @Override
    public ResponseEntity<List<GOST>> getBooks(Principal principal)
    {
        if (principal != null) {
            log.info("User {} запросил весь список литературы", principal.getName());
        }
        return ResponseEntity.ok(GostWorker.ShowAll());
    }
}
```

4. **Выводы**

В процессе лабораторной работы были получены практические навыки использования Apache Kafka